

END-TO-END ELECTRICITY DEMAND FORECASTING USING DEEP LEARNING

CHAPTER 1 – INTRODUCTION

1.1 Project Overview

Electricity demand forecasting is the process of predicting future electricity consumption based on historical demand patterns. Accurate forecasting is important for power generation planning, grid management, energy distribution, and resource allocation.

This project develops an end-to-end electricity demand forecasting system using deep learning. Historical hourly electricity demand data is processed, transformed into time-series features, used to train different recurrent neural network architectures, and finally deployed through a Flask web application.

The project includes:

- Data understanding
- Data preprocessing
- Feature engineering
- Exploratory Data Analysis
- Baseline forecasting
- Deep learning model development
- Hyperparameter tuning
- Model comparison
- 30/60-day validation
- Flask deployment
- Interactive forecasting dashboard

CHAPTER 2 – DATA UNDERSTANDING

2.1 Dataset

The project uses the **Hourly Energy Consumption – PJM** dataset.

The main target variable is: **PJME_MW**

This represents hourly electricity demand in megawatts.

The dataset contains: **Datetime, PJME_MW**

where Datetime represents the timestamp of the electricity demand observation.

2.2 Loading the Dataset

The dataset is loaded using Pandas.

```
df = pd.readcsv("../Dataset/PJME_hourly.csv")
```

Why this code is used

`pd.read_csv()` is used to load the CSV dataset into a Pandas DataFrame.

The DataFrame provides functions for:

- Data inspection
- Data cleaning
- Feature creation
- Statistical analysis
- Time-series processing

The path points to the dataset stored inside the project directory.

2.3 Converting Datetime

```
df["Datetime"] = pd.to_datetime(df["Datetime"])
```

Why this code is used

The Datetime column must be converted from a text/string representation into Pandas datetime format.

This is necessary because later feature engineering operations require datetime information.

For example, we need to extract:

- Hour
- Day
- Month
- Year
- Day of week
- Week number

2.4 Chronological Sorting

```
df = df.sort_values("Datetime")
```

Why this code is used

Electricity demand is a **time-series problem**.

Therefore, observations must remain in chronological order.

For example:

January → February → March → ... → December

Randomly mixing the observations could allow future information to appear before past information and cause data leakage.

CHAPTER 3 – DATA PREPROCESSING AND FEATURE ENGINEERING

3.1 Time-Based Feature Creation

The following features were extracted from the timestamp:

```
df["Hour"] = df["Datetime"].dt.hour
```

```
df["Day"] = df["Datetime"].dt.day
```

```
df["DayOfWeek"] = (  
    df["Datetime"].dt.dayofweek  
)
```

```
df["Week"] = (  
    df["Datetime"]  
    .dt.isocalendar()  
    .week  
    .astype(int)  
)
```

```
df["Month"] = df["Datetime"].dt.month
```

```
df["Year"] = df["Datetime"].dt.year
```

Why these features are created

Electricity demand is strongly influenced by time.

For example:

- Demand at 3 AM can differ from demand at 7 PM.
- Weekdays can have different demand from weekends.
- Summer and winter months can have different consumption patterns.

Therefore, these features provide temporal information to the model.

3.2 Weekend Feature

```
df["IsWeekend"] = (  
    df["DayOfWeek"] >= 5  
) .astype(int)
```

Why this code is used

DayOfWeek uses:

0 → Monday
1 → Tuesday
2 → Wednesday
3 → Thursday
4 → Friday
5 → Saturday
6 → Sunday

Therefore:

```
df["DayOfWeek"] >= 5
```

identifies Saturday and Sunday.

The result is converted into:

0 → Weekday
1 → Weekend

This allows the model to learn different electricity demand behavior during weekends.

4. LAG FEATURE ENGINEERING

4.1 Lag Variables

The project creates several lag variables:

```
df["Lag_1"] = df["PJME_MW"].shift(1)  
df["Lag_2"] = df["PJME_MW"].shift(2)  
df["Lag_3"] = df["PJME_MW"].shift(3)  
df["Lag_6"] = df["PJME_MW"].shift(6)  
df["Lag_12"] = df["PJME_MW"].shift(12)  
df["Lag_24"] = df["PJME_MW"].shift(24)
```

```
df["Lag_48"] = df["PJME_MW"].shift(48)
df["Lag_72"] = df["PJME_MW"].shift(72)
df["Lag_168"] = df["PJME_MW"].shift(168)
```

Why lag features are used

Electricity demand has strong temporal relationships.

For example:

Lag_1

represents the previous hour's demand.

Lag_24

represents demand approximately 24 hours earlier.

Lag_168

represents demand approximately one week earlier.

These historical values provide the model with information about previous demand patterns.

5. ROLLING STATISTICS

5.1 Rolling Mean

```
df["RollingMean_24"] = (
    df["PJME_MW"]
    .rolling(24)
    .mean()
)
```

Why it is used

A 24-hour rolling mean represents the average demand over the previous 24 hours.

This helps smooth short-term fluctuations and provides information about the recent demand level.

5.2 Weekly Rolling Mean

```
df["RollingMean_168"] = (  
    df["PJME_MW"]  
    .rolling(168)  
    .mean()  
)
```

Why it is used

Since:

24 hours × 7 days = 168 hours

the 168-hour rolling mean captures approximately one week of demand behavior.

5.3 Rolling Standard Deviation

```
df["RollingStd_24"] = (  
    df["PJME_MW"]  
    .rolling(24)  
    .std()  
)
```

Why it is used

Rolling standard deviation measures recent demand variability.

A high value indicates that demand has been fluctuating significantly.

A low value indicates relatively stable demand.

6. HANDLING MISSING VALUES

After creating lag and rolling features, the beginning of the dataset contains missing values.

For example:

Lag_168

cannot be calculated for the first 168 observations.

Therefore:

```
df = df.dropna().reset_index(drop=True)
```

Why this code is used

Rows containing incomplete lag or rolling features are removed.

`reset_index(drop=True)` resets the DataFrame index after removing the rows.

The processed dataset is then saved:

```
df.to_csv(  
    "../Dataset/final_df.csv"  
)
```

This file becomes the main processed dataset used by the later stages of the project.

CHAPTER 4 – FEATURE SELECTION

4.1 Input Features

The final model uses 19 features:

```
features = [  
    "PJME_MW",  
    "Hour",  
    "Day",  
    "Week",  
    "Month",  
    "DayOfWeek",  
    "IsWeekend",  
    "Lag_1",  
    "Lag_24",  
    "Lag_168",  
    "RollingMean_24",  
    "RollingMean_168",  
    "RollingStd_24",  
    "Lag_2",  
    "Lag_3",  
    "Lag_6",  
    "Lag_12",  
    "Lag_48",  
    "Lag_72"  
]
```

The target is = **"PJME_MW"**

Then:

```
X = df[features]
```

```
y = df[target]
```

Why this code is used

X contains the information given to the neural network.

y contains the electricity demand that the model must learn to predict.

Therefore:

$X \rightarrow \text{Input}$, $y \rightarrow \text{Output / Target}$

CHAPTER 5 – DATA SPLITTING

5.1 Chronological Train, Validation and Test Split

The dataset is divided chronologically.

```
train_size = int(len(df) * 0.70)
```

```
val_size = int(len(df) * 0.15)
```

Then:

```
X_train = X.iloc[:train_size]
```

```
X_val = X.iloc[
    train_size:
    train_size + val_size
]
```

```
X_test = X.iloc[
    train_size + val_size:
]
```

The same process is applied to y .

Why chronological splitting is used

For time-series forecasting, random splitting is inappropriate because it can introduce future information into the training data.

The chronological approach creates:

Past -----> Future

Training → Validation → Testing

This better represents how the model would operate in the real world.

CHAPTER 6 – FEATURE SCALING

6.1 StandardScaler

```
from sklearn.preprocessing import StandardScaler
```

```
X_scaler = StandardScaler()
```

Training data is used to fit the scaler:

```
X_train_scaled = X_scaler.fit_transform(  
    X_train  
)
```

Validation and test data are transformed using the same scaler:

```
X_val_scaled = X_scaler.transform(  
    X_val  
)
```

```
X_test_scaled = X_scaler.transform(  
    X_test  
)
```

Why scaling is used

The 19 features have different numerical ranges.

For example:

Hour → 0–23

Month → 1–12

PJME_MW → thousands of MW

Without scaling, larger numerical values could dominate the training process.

Standardization transforms features into a comparable numerical range.

6.2 Saving the Input Scaler

```
joblib.dump(  
    X_scaler,  
    "../Models/X_scaler.pkl"  
)
```

Why this code is used

The Flask application must preprocess future input exactly the same way as the training pipeline.

Therefore, the trained scaler is saved and later loaded by the Flask application.

7. TARGET SCALING

```
y_scaler = StandardScaler()
```

The target is scaled:

```
y_train_scaled = y_scaler.fit_transform(  
    y_train.values.reshape(-1, 1)  
)
```

Validation and test targets are transformed:

```
y_val_scaled = y_scaler.transform(  
    y_val.values.reshape(-1, 1)  
)
```

```
y_test_scaled = y_scaler.transform(  
    y_test.values.reshape(-1, 1)  
)
```

The scaler is saved:

```
joblib.dump(  
    y_scaler,  
    "../Models/y_scaler.pkl"  
)
```

Why target scaling is used

Neural networks generally train more effectively when the target values are normalized.

After prediction, the values must be converted back into MW:

```
y_scaler.inverse_transform(...)
```

CHAPTER 8 – SEQUENCE CREATION

8.1 Sequence Creation Function

```
def create_sequences(
    X,
    y,
    sequence_length,
    forecast_horizon
):

    X_sequences = []
    y_sequences = []

    for i in range(
        sequence_length,
        len(X) - forecast_horizon + 1
    ):

        X_sequences.append(
            X[
                i-sequence_length:i
            ]
        )

        y_sequences.append(
            y[
                i:i+forecast_horizon
            ]
        )

    return (
        np.array(X_sequences),
        np.array(y_sequences)
    )
```

Why this function is used

Recurrent neural networks require sequential input.

Instead of giving the model individual rows, historical observations are grouped into sequences.

For example, with:

sequence_length = 48

forecast_horizon = 24

the model receives:

Previous 48 hours

↓

Predict next 24 hours

CHAPTER 9. DEEP LEARNING MODEL DEVELOPMENT

The project evaluates four recurrent architectures:

Simple RNN

LSTM

GRU

Bidirectional LSTM

These models are appropriate because electricity demand contains temporal dependencies.

CHAPTER 10. RNN MODEL

The Simple RNN is used as the basic recurrent baseline.

Purpose

It provides a simple recurrent architecture against which more advanced models can be compared.

The RNN learns relationships between sequential observations.

CHAPTER 11. LSTM MODEL

LSTM is designed to handle longer-term dependencies in sequential data.

Purpose

Electricity demand can depend on patterns from previous hours and days.

LSTM uses memory mechanisms to preserve important information across time steps.

CHAPTER 12. GRU MODEL

GRU is another recurrent architecture designed to capture temporal relationships.

Purpose

GRU provides a computationally simpler alternative to LSTM while still being capable of learning sequential dependencies.

CHAPTER 13. BIDIRECTIONAL LSTM

The project also uses Bidirectional LSTM.

Purpose

Bidirectional LSTM processes the sequence in both directions during sequence encoding.

It provides another architecture for learning temporal relationships and was experimentally found to perform strongly in this project.

CHAPTER 14 – SEQUENCE LENGTH EXPERIMENT

Three historical sequence lengths were evaluated:

24 hours

48 hours

168 hours

Why different sequence lengths are tested

Different sequence lengths provide different amounts of historical information.

24 hours

→ One day of historical context

48 hours

→ Two days of historical context

168 hours

→ One week of historical context

This experiment determines how much historical information is most useful for the forecasting task.

CHAPTER 15. MODEL EVALUATION

The models were evaluated using:

MAE

RMSE

MAPE

R^2

Bias

15.1 MAE

Mean Absolute Error measures the average absolute forecasting error.

Lower MAE = Better

15.2 RMSE

Root Mean Squared Error gives more importance to large errors.

Lower RMSE = Better

15.3 MAPE

Mean Absolute Percentage Error expresses prediction error as a percentage.

Lower MAPE = Better

15.4 R^2

R^2 measures how well the model explains the variation in demand.

Higher R^2 = Better

15.5 Bias

Bias measures the average difference between actual and predicted values.

Bias = mean(actual - predicted) A value close to zero indicates less systematic overprediction or underprediction.

CHAPTER 16. EXPERIMENTAL RESULTS

Sequence Length = 24

Model	MAE	RMSE	MAPE	R^2	Bias
RNN	2846.42	3783.65	9.48	0.6564	947.41
LSTM	3283.34	4331.46	11.14	0.5498	2062.53
GRU	4117.81	5192.92	14.13	0.3528	2411.61
Bi-LSTM	2695.59	3642.70	8.97	0.6816	1326.54

Sequence Length = 48

Model	MAE	RMSE	MAPE	R^2	Bias
RNN	3113.50	4235.22	10.39	0.5699	951.41
LSTM	3116.87	4212.93	10.46	0.5744	1788.96
GRU	3431.37	4571.75	11.60	0.4989	1739.28
Bi-LSTM	2483.91	3341.67	8.35	0.7323	1040.45

Sequence Length = 168

Model	MAE	RMSE	MAPE	R^2	Bias
RNN	2505.47	3429.29	8.37	0.7188	1325.38
LSTM	3084.03	4103.47	10.45	0.5973	1673.60
GRU	3645.55	4866.68	12.34	0.4336	2286.38

Bi-LSTM	2600.27	3471.09	8.81	0.7119	1223.35
---------	---------	---------	------	--------	---------

Result

The 168-hour Bi-LSTM produced the strongest overall result among the reported experiments.

Therefore, this configuration was selected for the forecasting application.

CHAPTER 17. HYPERPARAMETER TUNING

Hyperparameter tuning was performed to identify a better model configuration.

For example:

```
hp.Choice(  
    "rnn_units",  
    values=[32, 64, 128]  
)
```

and:

```
hp.Choice(  
    "dense_units",  
    values=[32, 64, 128]  
)
```

Learning rate:

```
hp.Choice(  
    "learning_rate",  
    values=[  
        0.0001,  
        0.0005,  
        0.001  
    ]  
)
```

Why hyperparameter tuning is used

The architecture and training behavior of a neural network depend on parameters such as:

- Number of neurons
- Dense layer size
- Learning rate

Instead of manually guessing these values, Keras Tuner evaluates different combinations.

CHAPTER 18. RANDOM SEARCH

```
tuner = kt.RandomSearch(  
    build_rnn,  
    objective="val_loss",  
    max_trials=5,  
    executions_per_trial=1,  
    directory="hyperparameter_tuning",  
    project_name="rnn_24_to_24"  
)
```

Why Random Search is used

Random Search explores different combinations of hyperparameters.

The objective:

`val_loss`

means that configurations producing lower validation loss are preferred.

CHAPTER 19. FINAL MODEL

After tuning:

```
best_hp = tuner.get_best_hyperparameters(  
    num_trials=1  
)[0]
```

The best model is retrieved:

```
best_model = tuner.get_best_models(  
    num_models=1  
)[0]
```

The model can then be trained and saved.

CHAPTER 20 – PHASE 7: 30/60-DAY VALIDATION

The final forecasting evaluation uses recent unseen periods.

The data must remain strictly chronological.

The validation strategy separates recent data before final model evaluation.

The purpose is to answer:

How well does the trained model perform when forecasting a recent period that it has not seen during training?

Two validation periods are provided:

30 Days

60 Days

CHAPTER 21. WHY 30/60-DAY VALIDATION IS USED

A model can achieve good test metrics but still perform poorly on recent demand patterns.

Therefore, recent-period validation provides a more realistic evaluation.

The validation process compares:

Actual Demand

↓

Model Prediction

↓

Forecast Error

CHAPTER 22. VALIDATION VISUALIZATIONS

The validation page contains:

Actual vs Forecast

Shows whether the model follows the actual electricity demand curve.

Daily Error

Shows the forecasting error for each day.

Peak Demand Error

Examines model performance during high-demand periods.

Forecast Error Distribution

Shows whether errors are centered around zero or biased toward positive/negative values.

CHAPTER 23 – FLASK APPLICATION

23.1 Purpose

The trained model is deployed using Flask.

The purpose of the Flask application is to provide a user-friendly interface instead of requiring users to execute Python code manually.

CHAPTER 24. APPLICATION STRUCTURE

```
flask_app/  
|  
├── app.py  
|  
├── templates/
```



```
| |— base.html
| |— index.html
| |— forecast.html
| |— analytics.html
| |— validation.html
|
|— static/
|   |— css/
|   |— js/
```

CHAPTER 25. BASE TEMPLATE

base.html contains common components such as:

- Navigation bar
- Bootstrap
- Common styling
- JavaScript libraries
- Page structure

Other pages use:

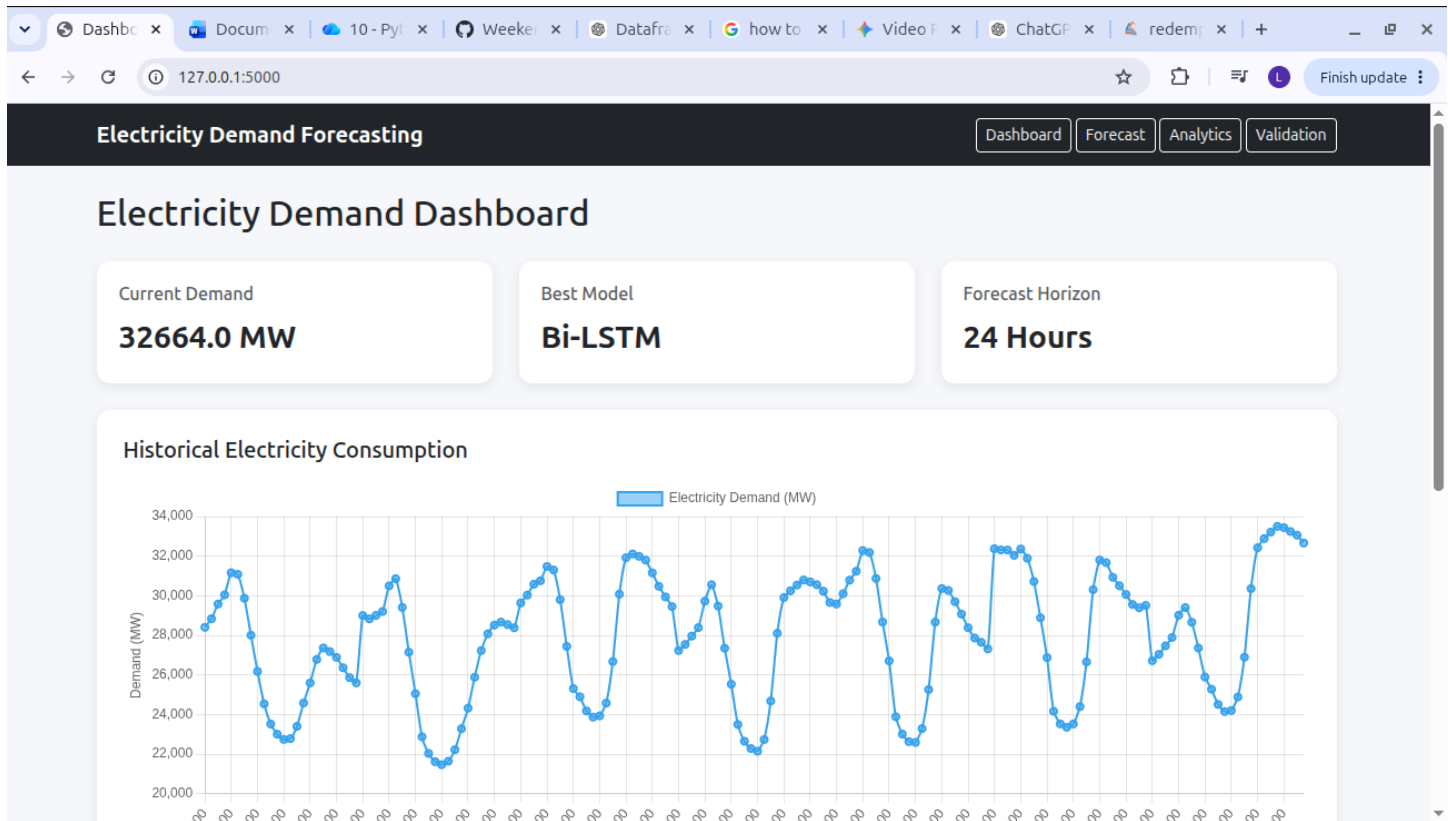
```
{% extends "base.html" %}
```

Why this is used

Without template inheritance, the same navigation and styling code would need to be repeated in every HTML file.

Template inheritance makes the application:

- Cleaner
- Easier to maintain
- More consistent



Dashboard Page

CHAPTER 26. FORECAST ROUTE

The main forecasting route is:

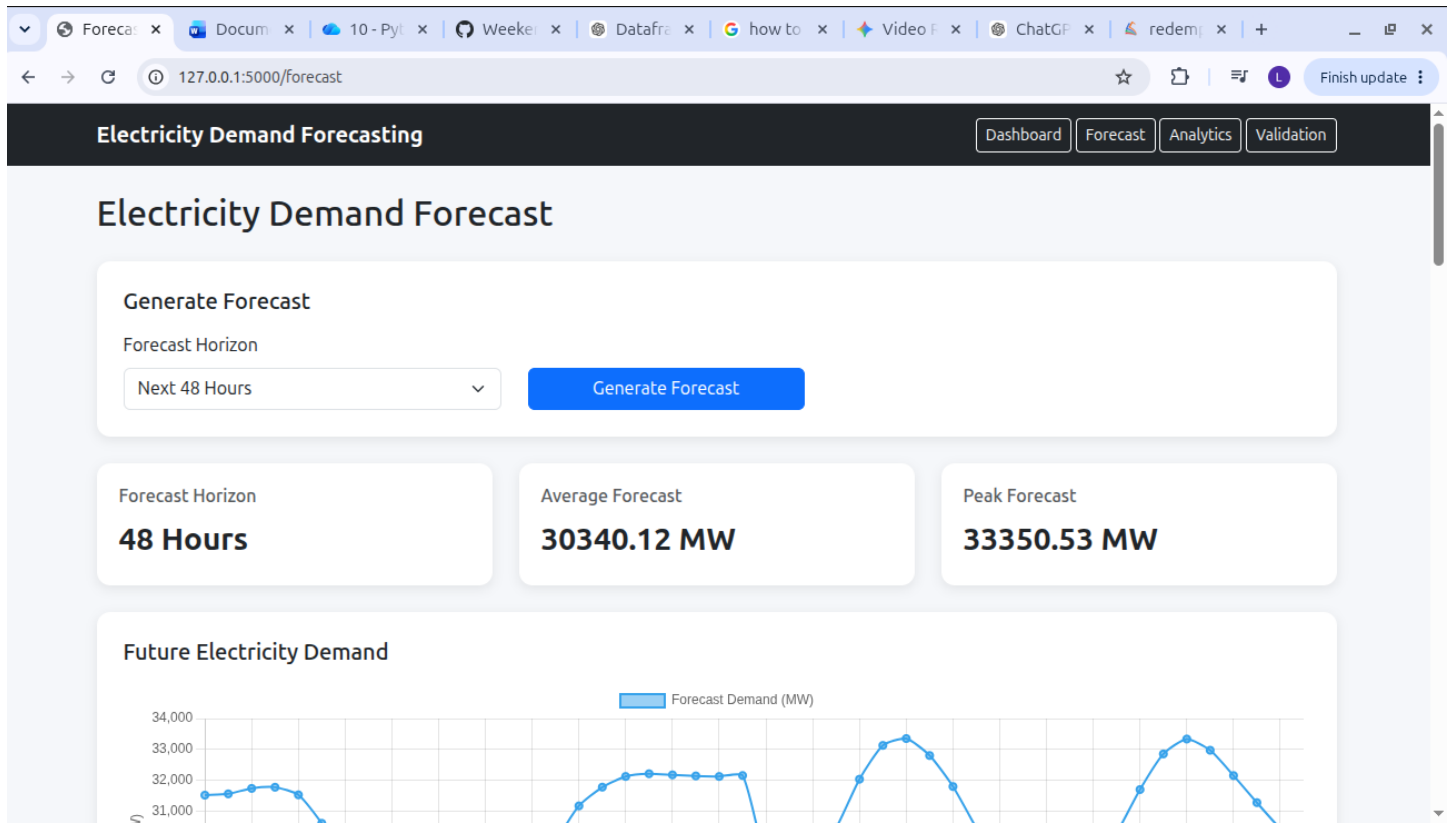
```
@app.route(
    "/forecast",
    methods=["GET", "POST"])
def forecast_page():
```

Why this route is used

The page must support:

GET → Display forecasting page

POST → Generate forecast



forecast Page

CHAPTER 27. LOADING THE PROCESSED DATA

```
df = pd.read_csv(
    "../Dataset/final_df.csv"
)

df["Datetime"] = pd.to_datetime(
    df["Datetime"]
)

df = df.sort_values(
    "Datetime"
).reset_index(drop=True)
```

Why this code is used

The Flask application must use the same processed features used during model training.

The data is sorted chronologically to make sure the latest observations are correctly identified.

CHAPTER 28. SELECTING THE LATEST SEQUENCE

```
latest_features = df[
    features
].tail(48)
```

Why this code is used

The selected model was trained using a 48-hour historical sequence.

Therefore, the application takes the latest 48 observations as input.

The resulting data contains:

48 time steps
×
19 features

CHAPTER 29. LOADING THE SCALERS

```
X_scaler = joblib.load(
    "../Models/X_scaler.pkl"
)
```

```
y_scaler = joblib.load(
    "../Models/y_scaler.pkl"
)
```

Why this code is used

The same scalers used during training must be used during prediction.

Otherwise, the input distribution seen by the model would be different from the training distribution.

CHAPTER 30. SCALING THE INPUT

```
latest_scaled = X_scaler.transform(  
    latest_features  
)
```

Why this code is used

The latest input must be transformed using the training scaler before being passed to the neural network.

CHAPTER 31. RESHAPING THE INPUT

```
X_input = latest_scaled.reshape(  
    1,  
    48,  
    19  
)
```

The model expects:

Samples × Time Steps × Features

Therefore:

$1 \times 48 \times 19$

means:

1 sample

48 historical hours

19 features

CHAPTER 32. GENERATING THE FORECAST

```
prediction_scaled = model.predict(  
    X_input,  
    verbose=0  
)
```

Why this code is used

The trained Bi-LSTM receives the latest historical sequence and generates the predicted future demand.

CHAPTER 33. CONVERTING PREDICTIONS BACK TO MW

```
prediction = (  
    y_scaler  
    .inverse_transform(  
        prediction_scaled.reshape(-1, 1)  
    )  
    .flatten()  
)
```

Why this code is used

The model predicts values in the scaled target space.

Users need predictions in the original unit:

MW

Therefore, `inverse_transform()` converts the predictions back to their original scale.

CHAPTER 34. GENERATING FUTURE TIMESTAMPS

```
forecast_dates = pd.date_range(  
    start=last_timestamp  
    + pd.Timedelta(hours=1),  
    periods=len(forecast),  
    freq="h"  
)
```

Why this code is used

The predictions represent future hourly demand.

Therefore, the application generates corresponding future timestamps.

For example:

Last historical observation

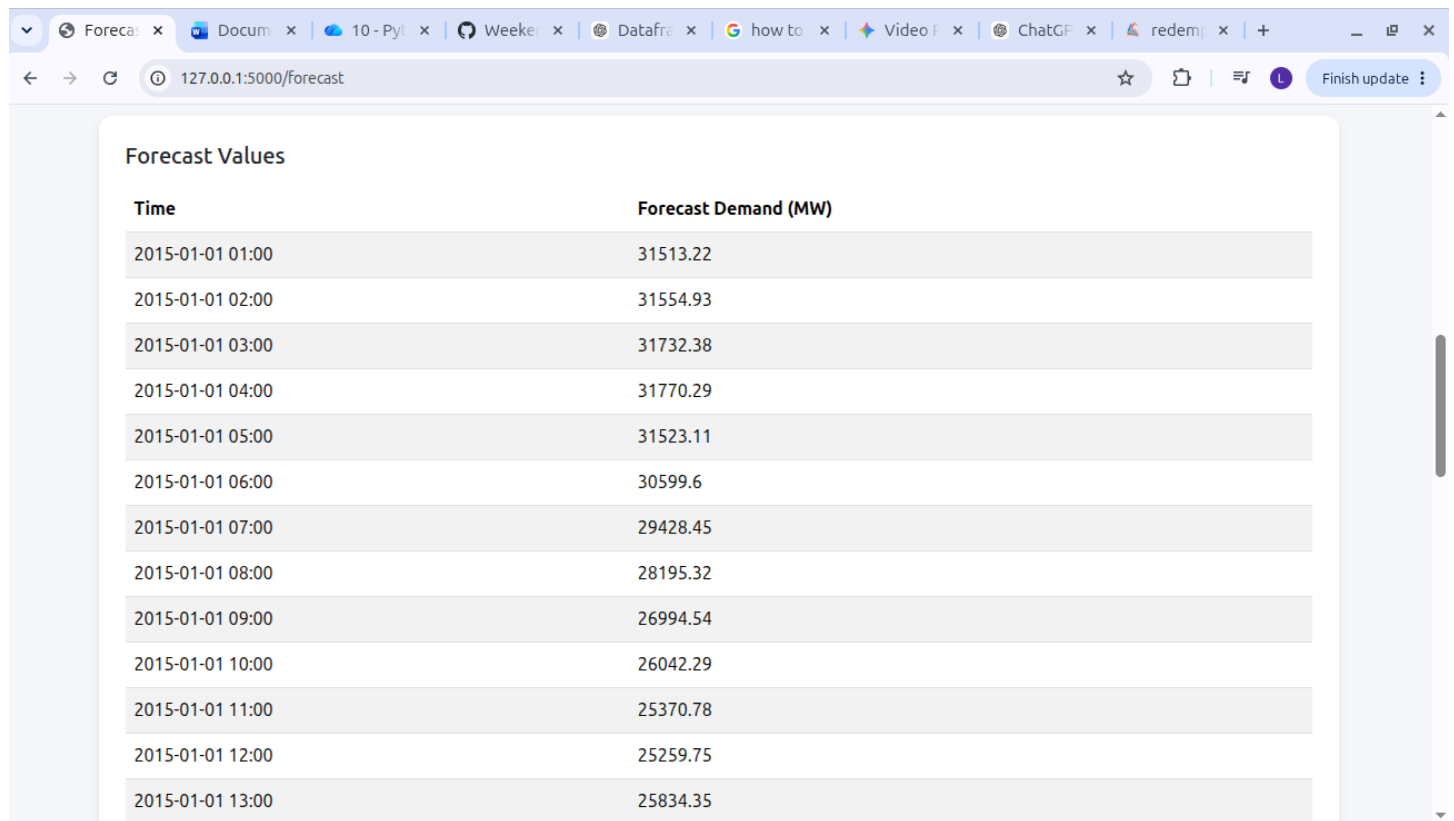


+1 hour

+2 hours

+3 hours

...

A screenshot of a web browser window. The address bar shows '127.0.0.1:5000/forecast'. The page title is 'Forecast Values'. It contains a table with two columns: 'Time' and 'Forecast Demand (MW)'. The table lists 13 rows of data, starting from '2015-01-01 01:00' and ending at '2015-01-01 13:00'. The demand values range from 31513.22 to 25834.35.

Time	Forecast Demand (MW)
2015-01-01 01:00	31513.22
2015-01-01 02:00	31554.93
2015-01-01 03:00	31732.38
2015-01-01 04:00	31770.29
2015-01-01 05:00	31523.11
2015-01-01 06:00	30599.6
2015-01-01 07:00	29428.45
2015-01-01 08:00	28195.32
2015-01-01 09:00	26994.54
2015-01-01 10:00	26042.29
2015-01-01 11:00	25370.78
2015-01-01 12:00	25259.75
2015-01-01 13:00	25834.35

Future Forecast

CHAPTER 35. FORECAST SUMMARY

The application calculates:

```
average_forecast = round(  
    float(np.mean(forecast)),  
    2  
)
```

and:

```
peak_forecast = round(  
    float(np.max(forecast)),
```

Why these are used

The user can quickly understand the forecast without examining every individual value.

The dashboard displays:

Forecast Horizon

Average Forecast

Peak Forecast

CHAPTER 36. FORECAST PAGE

The Forecast page provides:

- Forecast horizon selection
- Generate Forecast button
- Forecast summary
- Forecast chart
- Forecast table

The user can select:

24 Hours

48 Hours

72 Hours

and generate the corresponding forecast.

CHAPTER 37. ANALYTICS PAGE

The Analytics page provides:

Latest Demand

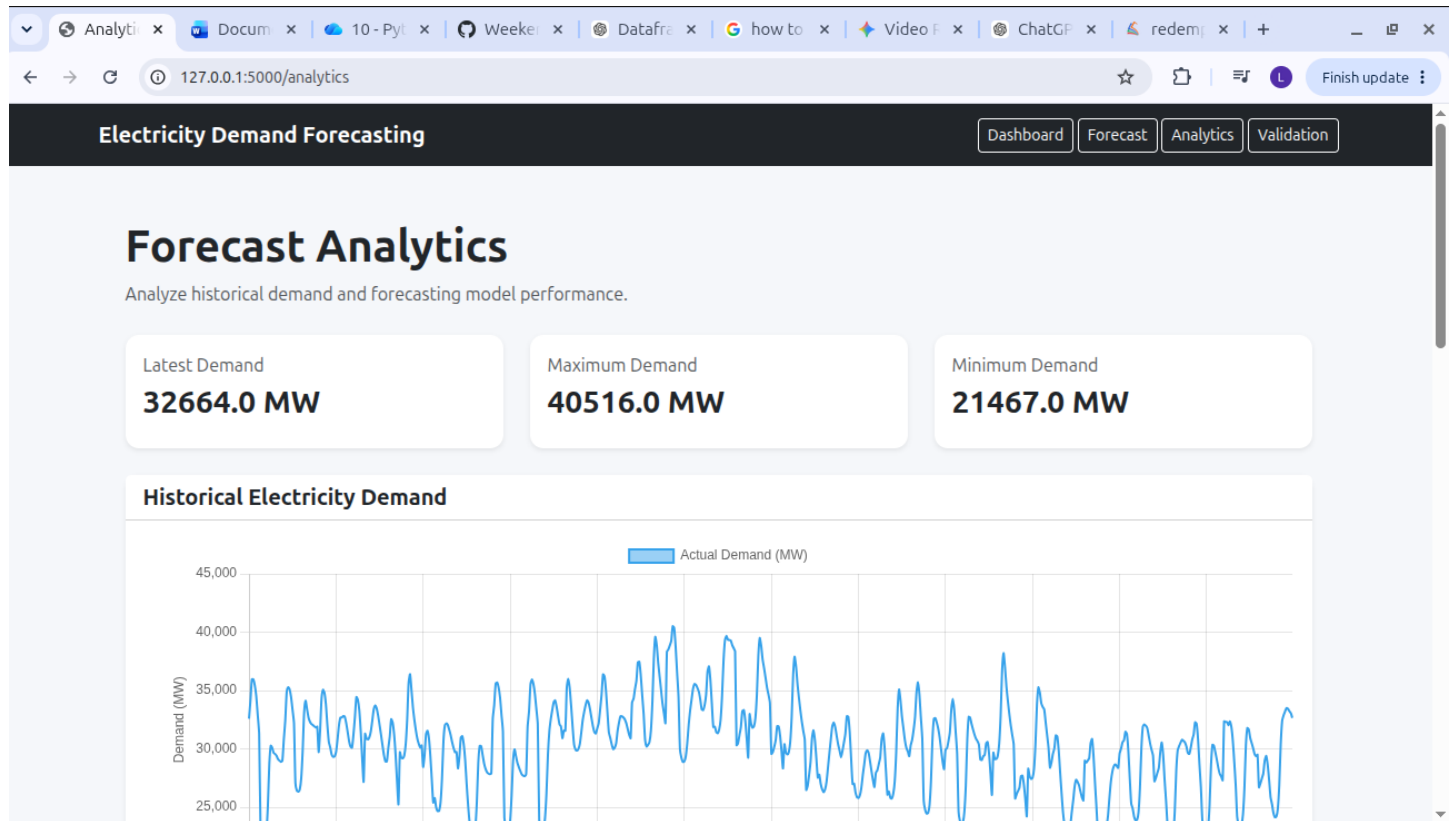
Maximum Demand

Minimum Demand

Historical Demand

Model Comparison
Forecast Error
Peak Demand Analysis
Forecast Performance

This page is designed to help users understand model behavior rather than simply view future predictions.



Analytics Page

CHAPTER 38. VALIDATION PAGE

The validation page provides a dedicated interface for:

30-Day Validation
60-Day Validation

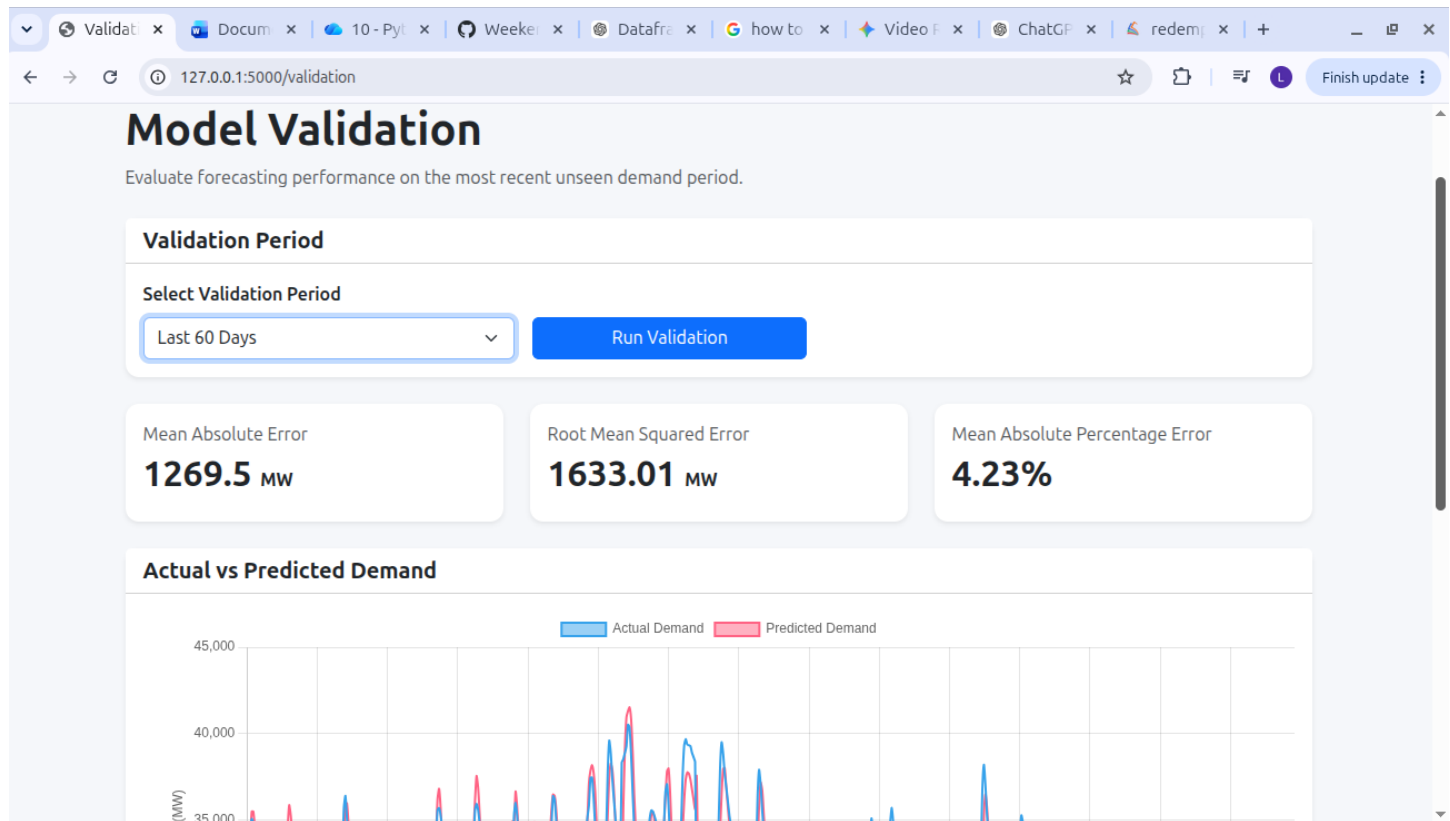
The user can compare:

Actual Demand
vs
Predicted Demand

and view:

MAE
RMSE
MAPE
 R^2

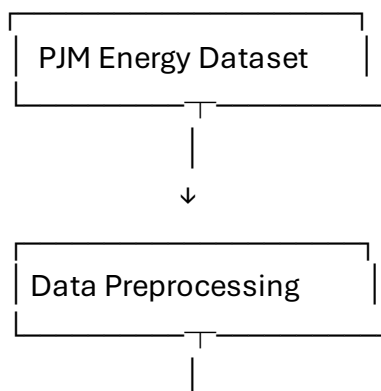
along with graphical performance analysis.

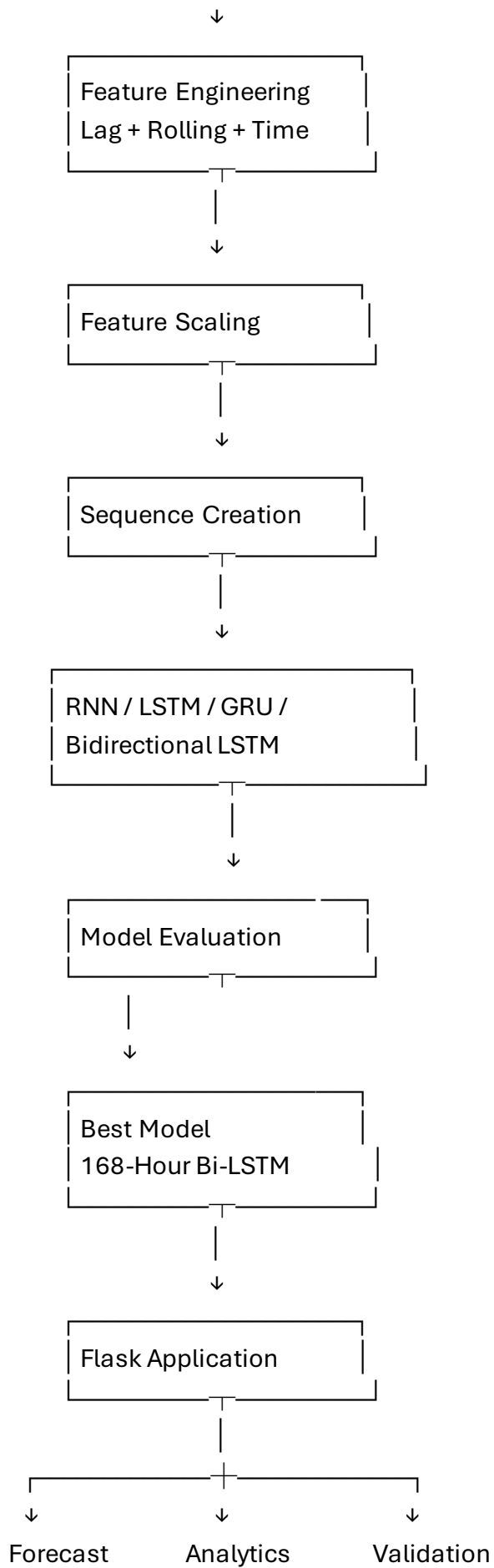


Validation Page

CHAPTER 39. SYSTEM ARCHITECTURE

The complete system can be represented as:





CHAPTER 40. RESULTS AND DISCUSSION

The experiments demonstrate that recurrent deep learning models can learn meaningful temporal patterns from historical electricity demand.

Among the tested architectures, the Bidirectional LSTM produced strong results, particularly with the 168-hour input sequence.

The 168-hour Bi-LSTM achieved:

Metric	Result
MAE	1716.29 MW
RMSE	2290.09 MW
MAPE	5.67%
R ²	0.8747
Bias	216.42 MW

The experiment with different sequence lengths also demonstrates that increasing the historical window does not automatically improve forecasting performance. The 168-hour sequence performed better overall than the tested 24-hour and 48-hour configurations for the selected Bi-LSTM model.

CHAPTER 41. CONCLUSION

The project successfully developed an end-to-end electricity demand forecasting system using deep learning.

The project began with raw historical electricity demand data and progressed through preprocessing, temporal feature engineering, lag generation, rolling statistics, exploratory analysis, baseline forecasting, sequence creation, deep learning model development, hyperparameter tuning, and model evaluation.

Four recurrent architectures were compared:

RNN

LSTM

GRU

Bi-LSTM

Different historical sequence lengths were also evaluated. Based on the reported experiments, the **48-hour Bi-LSTM** was selected as the main forecasting model.

Finally, the model was integrated into a Flask web application containing:

Dashboard

Forecast

Analytics

Validation

The resulting system provides both **future electricity demand predictions** and **visual analytical tools** for evaluating forecasting performance.